Chapitre 5 : Abstraction

L'objectif de cette fiche est d'introduire le mécanisme d'abstraction en Java, complémentaire de l'héritage, ainsi que la notion d'interface.

Notion de classe et de méthode abstraite

Un Pangolin, c'est très joli, mais nous avons bien assez fait crier ces petites bêtes. Qu'elles se reposent un peu.

Dans cette fiche, on s'appuie sur un grand classique de la POO : on suppose qu'on veut écrire un programme qui manipule des **figures géométriques** de plusieurs types : des cercles, des rectangles par exemple.

Chaque figure est caractérisée par un certain nombre de propriétés, qui vont se concrétiser dans le code par des attributs. Ainsi, toutes les figures ont un point central ; les cercles ont en plus un rayon ; et les rectangles ont en plus une largeur et une hauteur.

Chaque figure possède également un certain nombre de savoir-faire, qui vont se concrétiser dans le code par des méthodes. Ainsi, par exemple, nos cercles et nos rectangles (bref : toutes nos figures) peuvent être translatés, et il est possible de calculer leur périmètre et leur surface.

Le concept d'héritage conduit bien sûr à introduire une classe mère `Figure` et autant de classes filles que de types de figures :

La classe mère `Figure`, s'impose en POO dans notre contexte. Son existence offre plusieurs avantages :

- la classe `Figure` factorise le code (attributs, méthodes) commun à toutes les figures
- elle traduit en Java le fait que tous les cercles et tous les rectangles SONT des cas particuliers d'une notion plus générique : les figures.
- ce faisant, elle rend possible l'usage du polymorphisme d'objets.

Par contre, à votre avis, est-il logique de pouvoir *instancier* des objets de type dynamique `Figure` ? Autrement dit, est-il légitime d'écrire quelque chose comme :

```
Figure f = new Figure(...);
```

Eh bien, non ! Dans notre petit programme, on instanciera des cercles et des rectangles, mais jamais de figure « tout court ». En effet, qu'est ce que

pourrait bien être une « figure » qui ne serait ni un rectangle, ni un cercle, ni un triangle, ni... ?

Ainsi, la **notion de Figure est un concept abstrait**. En POO, on **en fera de la classe `Figure` une classe abstraite** : la classe existe, mais on ne peut pas l'instancier.

Par ailleurs, que penser du code des méthodes `calculerSurface()` et `calculerPerimetre()` dans la classe `Figure` ? Certes, chaque figure (que ce soit un cercle ou un rectangle) peut calculer son périmètre... Mais bienheureux(se) celui ou celle qui peut écrire, par exemple, le code de la méthode `calculerPerimetre()` dans la classe `Figure` ! Car comment calculer le périmètre d'une « figure » si on ne sait pas si c'est un cercle ou un rectangle, ou ...?

Autrement dit, ces traitements sont des traitements qui existeront bien sur toutes les figures, mais auxquels on ne peut pas associer de comportement (de code) au niveau de la classe `Figure`. En POO, on on dira que ce sont des **méthodes abstraites** dans la classe `Figure` : elles sont déclarées dans la classe en tant que promesse de traitement qui existeront bien dans les sous-classes, mais sans code.

Classe abstraite en Java

Attention, cela va aller vite !

En Java, l'abstraction est dénotée par le mot clé `abstract`.

Une classe est déclarée abstraite de la façon suivante :

```
public abstract class MaClasse {
    [...]
}
```

Le seul effet du mot clé `abstract` appliqué à une classe est que cette classe ne peut plus être instanciée avec `new`. Il ne pourra donc jamais exister en mémoire un objet dont le type *dynamique* soit cette classe.

Ainsi, le code suivant provoquerait une erreur de compilation, car la classe `Figure` est abstraite :

```
Figure f = new Figure(...);
// Et le compilateur râle :
// "error: Figure is abstract; cannot be instantiated"
```

A part le fait qu'elle ne peut pas être instanciée, une classe abstraite s'écrit et se manipule comme toute autre classe. Elle peut contenir des attributs, des méthodes (dont des constructeurs), comme n'importe quelle classe.

Voici, par exemple, à quoi pourrait ressembler le début de notre classe `Figure` désormais abstraite :

```
public abstract class Figure {
    private double centreX;
    private double centreY;

    public Figure(double x, double y) {
        this.centreX = x;
        this.centreY = x;
    }
    [...]

    /** translate la figure du vecteur (dx, dy) */
    public void translater(double dx, double dy) {
        centreX += dx;
        centreY += dy;
    }
    [...]
}
```

Méthode abstraite

En Java, une méthode abstraite est une méthode :

- déclarée abstraite au moyen du mot clé `abstract`
- qui n'a pas de corps : on n'écrit que le prototype.

Une méthode abstraite peut être vue comme une promesse de traitement qui existera dans les sous-classes concrètes, mais dont on ne peut pas encore écrire le code dans la classe mère.

Voici à quoi ressemblerait les méthodes abstraites `calculerPerimetre()` et `calculerSurface()` dans la classe `Figure` :

```
public abstract class Figure {
    [...] // comme précédemment
    public abstract double calculerPerimetre(); // méthode abstraite
    public abstract double calculerSurface(); // méthode abstraite

}
```

Par ailleurs :

Toute classe qui déclare une (ou plusieurs) méthode(s) abstraite(s) est elle même nécessairement abstraite.

En conséquence, une classe qui a une méthode abstraite doit donc être elle-même déclarée `abstract` si vous voulez éviter que le compilateur ne râle. Rappelons qu'à l'inverse une classe peut être abstraite même si elle n'a pas de méthode abstraite.

Mais pourquoi s'embêter à déclarer des méthodes abstraites ?

Oui, tiens : pourquoi donc est-il important que la classe `Figure` ait une méthode abstraite `abstract double calculerPerimetre()` alors que cette méthode n'a pas de code ?

Voici trois explications :

- D'abord, du point de vue de la conception, la méthode `abstract double calculerPerimetre()` de la classe `Figure` traduit dans le code l'idée qu'il « est possible de calculer le périmètre de toutes les figures ». Il est donc logique qu'elle soit déclarée dans cette classe, même si on ne sait pas comment faire le calcul à ce niveau de la hiérarchie.
- Ensuite, une méthode abstraite est une promesse de traitement. Sa déclaration dans la super-classe va obliger à bien définir une version concrète dans les sous classes. Si par exemple dans la classe `Rectangle` on oubliait de définir cette méthode, alors cette méthode héritée resterait abstraite dans la sous-classe `Rectangle`. Mais alors, cette sous classe `Rectangle` serait elle même abstraite et on ne pourrait plus créer de `Rectangle` !
- Enfin, l'existence des méthodes abstraites dans la classe `Figure` rend possible l'usage de ces méthodes dans le cas du polymorphisme. En effet, le code suivant :

```
Figure f = new Rectangle(...);
double perim = f.calculerPerimetre();
...
```

ne compile et ne fonctionne que si `double calculerPerimetre()` est bien définie (même abstraite) dans la classe `Figure`. Si vous en doutez, vous pourrez avantageusement vous replonger dans la partie statique du mécanisme d'appel de méthode en Java de la fiche précédente...

Notez pour finir que dans la super-classe `Figure`, il serait illogique d'écrire une méthode `double getRayon()` (qu'elle soit abstraite ou non). En effet, la notion de « rayon » n'a de pas de sens au niveau de la super classe, mais uniquement dans la sous-classe `Cercle`, contrairement au périmètre par exemple (toute figure a un périmètre).

Un code Java pour nos Figures

Voici, avec l'abstraction, le schéma UML et un exemple de code pour nos classes (à compléter... par exemple avec des accesseurs). Notez qu'en UML l'abstraction est dénotée par la mise en italique, ou l'ajout du mot-clef `{abstract}` entre accolades.

```
public abstract class Figure {
    private double centreX;
    private double centreY;

    public Figure() {
        this.centreX = 0;
        this.centreY = 0;
    }

    public Figure(double x, double y) {
        this.centreX = x;
        this.centreY = x;
    }
```

```
    /** translate la figure du vecteur (dx, dy) */
    public void translater(double dx, double dy) {
        centreX += dx;
        centreY += dy;
    }

    public abstract double calculerPerimetre(); // méthode abstraite
    public abstract double calculerSurface(); // méthode abstraite


    @Override
    public String toString() {
        return "centre (" + x + "," + y + ")";
    }
}
import java.math.*;
public class Cercle extends Figure {
    private double rayon;

    /** constructeur par défaut : cercle de rayon 1 centré sur l'origine */
    public Cercle() {
        super();
        this.rayon = 1;
    }

    public Cercle(double x, double y, double rayon) {
        super(x, y);
        this.rayon = rayon;
    }

    public double getRayon() {
        return rayon;
    }

    /* définition des méthodes calculerPerimetre  et calculerSurface */
    @Override
    public double calculerPerimetre() {
        return 2 * Math.PI * rayon ;
    }

    @Override
    public double calculerSurface(){
        return Math.PI * rayon * rayon;
    }

    @Override
    public String toString() {
        return "Cercle " + super.toString() + " ; rayon = " + rayon;
    }
}
import java.math.*;
public class Rectangle extends Figure {
    private double largeur;
    private double hauteur;

    /** constructeur par défaut : rectangle de hauteur et largeur 1 centré
sur l'origine */
    public Cercle() {
        super();
        largeur = 1;
        hauteur = 1;
    }

    public Cercle(double x, double y, double hauteur, double largeur) {
        super(x, y);
        this.largeur = largeur;
        this.hauteur = hauteur;
    }
```

```
    /* définition des méthodes calculerPerimetre  et calculerSurface */
    @Override
    public double calculerPerimetre() {
        return 2*(hauteur+largeur) ;
    }

    @Override
    public double calculerSurface(){
        return largeur * hauteur;
    }

    @Override
    public String toString() {
        return "Rectangle " + super.toString() + " ; largeur = " + largeur
+ " hauteur = " + hauteur;
    }
}
```

Et une classe de test pour finir :

```
public class Test {
    public static void main(String [] args) {
        // Figure = new Figure (1, 2);
        // => refusé à la compilation, car Figure est une classe abstraite
        // et ne peut être instanciée !

        Cercle c = new Cercle();
        Rectangle r = new Rectangle(1, 2, 10, 12);

        c.translater(2, 2);

        System.out.println(c);
        System.out.println(r);

        System.out.println("** Perimetre de c = " + c.calculerPerimetre());
        System.out.println("** Surface de r = " + r.calculerSurface());

        // Ce qui précède affiche :
        // Cercle centre (2,2) ; rayon = 1
        // Rectangle centre (1,2) ; largeur = 10 hauteur = 12
        // ** Perimetre de c = 6.283185
        // ** Surface de r = 120

        // un peu de polymorphisme pour finir...
        Figure [] tab = new Figure[2];
        tab[0] = new Cercle();
        tab[1] = new Rectangle(1, 2, 3, 4);
        for(Figure f: tab) {
            System.out.println(f);
        }
        // Affiche :
        // Cercle centre (0,0) ; rayon = 1
        // Rectangle centre (1,2) ; largeur = 3 hauteur = 4
    }
}
```

Notion d'interface

En première approche, en java et plus généralement en POO, une interface est un type (tout comme une classe) qui regroupe un ensemble de méthodes abstraites dont on ne donne que la signature (sans code).

Une interface s'écrit comme une classe, mais au moyen du mot clé `interface` en lieu et place du mot clé `class`.

Une interface ne contient aucun traitement véritable. Elle se contente de déclarer un cadre pour un ensemble de traitements qui devront être implémentés plus tard par une classe. Une interface est donc destinée à être **réalisée** (on dit aussi **implémentée**) par des classes.

Une classe déclare qu'elle implémente (ou « réalise ») une interface au moyen du mot clé `implements`.

Une classe qui déclare implémenter une interface s'engage à fournir le service (le « contrat ») spécifié par l'interface. Elle doit donc fournir une implémentation pour chacune des méthodes listées dans l'interface.

Détails
En fait, ce n'est le cas que si la classe concrète. Une classe *abstraite* qui implémente une interface n'est pas tenue de fournir une implémentation pour toutes les méthodes déclarées dans l'interface. Mais ses sous-classes concrètes devront le faire, bien sûr (sans quoi elles ne seraient pas concrètes !).

Voyons un premier exemple, avec une interface `Deplacable`. Remarquez que le corps d'une interface se présente d'abord comme une classe allégée qui ne contient que des constantes et des signatures de méthodes (donc des méthodes abstraites).

```
/**Cette interface spécifie la notion abstraite d'objet déplaçable.
Toute classe qui implémente l'interface Deplacable
doit pouvoir retourner une position (en 2D)
et fournir des services de modification de la position */
interface Deplacable {
    double getX();
    double getY();
    void positionner(double x, double y);
    void translater(double dx, double dy);
}

// Et voici une version (partielle...) de la classe Point qui réalise
l'interface Deplacable
public class Point implements Deplacable {
    private double x;
    private double y;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    // Puisque la classe Figure réalise l'interface Deplacable
    // il faut que toutes les méthodes déclarée dans l'interface
    // soit implémentées par la classe
    @Override
    public double getX() {
        return this.x;
    }
    @Override
    public double getY() {
        return this.y;
    }
    @Override
    public void positionner(double x, double y) {
        this.x = x;
        this.y = y;
```

```
    }
    @Override
    public void translater(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }

    // on peut bien sûr munir la classe d'autres méthodes que celles
déclarées dans l'interface :
    void symetrieOrigine() {
        x = -x;
        y = -y;
    }

    @Override
    public toString() {
        return "Point (" + x + ", " + y + ")";
    }
}
```

Et voici le diagramme UML de notre exemple :

Vous pouvez remarquer sur ce schéma la notation UML pour une interface et l'implémentation d'une interface. Ça ressemble beaucoup à l'héritage, non ? Mais notez le trait pointillé, qui se traduit par « implémente » ou « réalise ».

Une interface peut être vue :

- Comme une collection d'opérations abstraites, qui spécifie un service (un comportement) que les classes qui la réalisent doivent nécessairement implémenter. C'est pour cela que le nom d'une interface, par convention, se termine souvent par « able » (Deplacable, Cloneable, Comparable...)
- Mais aussi comme une classe « purement abstraite », qui ne contient que des méthodes abstraites et aucun attribut.

Sur ce dernier point, les notions d'interface et de classe purement abstraite sont d'ailleurs si proches que, dans certains langages objet, comme le C++ une interface n'est ni plus, ni moins qu'une classe purement abstraite.

Mais il existe une différence conceptuelle importante : alors qu'une classe (abstraite ou non) représente ce « que sont » des objets (leur « essence », leur « être »), une interface garantit juste que les classes qui la réalisent offrent le service déclaré dans l'interface, indépendamment de ce que la classe représente.

En java, il y a une autre différence importante :

En Java (et en UML), une différence essentielle entre les notions d'interface et de classe purement abstraite est qu'**une classe peut implémenter plusieurs interfaces**, alors qu'elle ne peut hériter que d'une unique classe (héritage simple).

On peut écrire par exemple :

```
class BonhommeDeNeige implements Deplacable, Cassable,
PouvantEtreDecoreDuneCarotte {
    [...]
}
```

Les interfaces et le problème de l'héritage multiple en Java

En Java, les interfaces sont l'une des solutions au problème de l'héritage multiple. Puisque l'héritage multiple n'existe pas en java (pour de bonnes raisons...), on a parfois recours à des interfaces pour l'approcher. Si, par exemple, on voulait créer une classe `VehiculeAmphibie`, on pourrait décider de la faire hériter d'une classe `VehiculeTerrestre` et de la faire implanter une interface `DeplacableSurEau`.

Une interface est un type !

Une interface définit un type, de la même manière qu'une classe.

On peut donc déclarer des références du type d'une interface.

Toute instance d'une classe qui implémente une interface peut être considérée comme étant du type de l'interface. Ou, dit autrement, une référence du type d'une interface (type statique) peut être utilisée pour manipuler des objets dont le type dynamique est n'importe quelle classe qui implémente cette interface.

Une interface peut donc être utilisée, comme n'importe quelle super classe, pour le polymorphisme. Par exemple :

```
Deplacable d = new Point(10, 12);
```

Si par exemple on écrit une interface `Dessinable` qui déclare des méthodes de dessin, et diverses classes qui réalisent cette interface, alors on peut manipuler les instances de ces classes « en tant que » `Dessinable` :

```
interface Dessinable {
    void dessiner(Graphics g);
    void setColor(Color c);
    [...]
}

class BonhommeDeNeige extends ElementDeDecors implements Dessinable {
    [...]
    void setColor(Color c) {
        this.color = c;
    }
    void dessiner(Graphics g) {
        g.drawOval(...);
        [...]
    }
    [...]
}

class Rectangle extends Figure implements Dessinable, Redimensionnable {
    [...]
    void setColor(Color c) {
        this.color = c;
    }
```

```
    void dessiner(Graphics g) {
        g.drawRect(...);
        [etc.]
    }
    [...]
}

class PangolinDessinable extends Pangolin implements Dessinable {
    [...]
    void setColor(Color c) {
        this.color = c;
    }
    void dessiner(Graphics g) {
        g.drawLine(...);
        [...]
    }
    [...]
}

class Test {
    public static void main(String args[]) {
        Graphics g = [...];
        // un tableau d'objets dessinable
        Dessinable[] tab = new Dessinable [3];
        // Avec ce tableau, on va pouvoir stocker des objets "en tant que
dessinable" :
        // Peu importe qu'ils soient en fait des instances (type dynamique)
de classes très diverses
        // (des bonhommes, des rectangles, des torchons et des
serviettes...)
        // dès lors que leur classe (type dynamique) réalise l'interface
Dessinable.

        tab[0] = new BonhommeDeNeige(...);
        tab[1] = new Rectangle(...);
        tab[2] = new PangolinDessinable(...);
        for(Dessinable d : tab) {
            d.dessiner(g);
        }
        // Et voila un beau dessin avec un Pangolin posé sur un Rectangle
et en émoi devant un BonhommeDeNeige !
    }
}
```

Notez que les règles du polymorphisme s'appliquent avec les interfaces de la même manière que pour les classes. En conséquence, comme d'habitude, sur une référence de type interface, seules les méthodes définies dans l'interface peuvent être exécutées :

```
Rectangle r = new Rectangle(2, 3, 12, 10);
r.translater(3, 4) ; // OK, pas de problème
Dessinable d = r; // on considère le rectangle "en tant qu'objet
dessinable"
d.setColor(Color.RED) ; // OK, pas de problème : setColor(Color) est
déclarée dans l'interface
d.translater(3, 4) ; // erreur de compilation, même si un Rectangle a bien
une méthode translater !
                     // car translater() n'est pas définie dans Dessinable
```

Résumons...

On résume ci après les éléments essentiels à connaître pour les interfaces.

- Une interface est une liste de méthodes dont on donne seulement la signature.

- Une interface ne peut pas déclarer d'attribut - plus précisément, mais ce n'est pas essentiel : une interface ne peut déclarer que des attributs constants, déclarés `static final`.
- Toutes les méthodes d'une interface sont implicitement déclarées `abstract` et `public`. Il n'est pas nécessaire d'utiliser ces qualificateurs dans une interface - mais c'est possible.
- Une interface est un « contrat » que « réalise » ou « implémente » des classes (mot clé `implements`).
- Alors qu'une classe ne peut hériter que d'une unique autre classe, elle peut implémenter autant d'interface qu'elle le souhaite.
- Des classes dans des hiérarchies différentes peuvent implémenter la même interface.
- **Héritage entre interfaces** : une interface peut hériter d'un nombre **quelconque** d'interfaces, avec le mot clé `extends`.
  Exemple : `interface SuperDessinable extends Coloriable, Dessinable.`

À noter... ou pas...
Notons que la version 8 de Java a modifié assez profondément la notion d'interface en ajoutant les notions de « méthodes statiques d'interface » et de « méthode par défaut », qui ne sont pas développées ici. Tout ce qui précède demeure bien sûr valable en Java 8.

Les classes anonymes

Voici une petite digression par une construction bien pratique de Java : les classes anonymes.

Revenons sur notre interface `Dessinable` de tout-à-l'heure. Pour pouvoir l'utiliser, nous avons besoin d'écrire explicitement le code de classes qui réalisent cette interface et de créer des objets de ces classes. Reprenons le code ci-avant. Le bonhomme de neige dessinable ne sert qu'une seule fois. Il est peut-être un peu dommage de créer une classe nommée juste pour cela, alors que finalement, nous n'avons besoin que d'un objet qui soit dessinable, et dont on redéfinit précisément le comportement à la volée.

Ça tombe bien, nous pouvons définir en Java une classe **anonyme** et s'en servir pour instancier une variable. Voyez plutôt l'exemple ci-dessous.

```
interface Dessinable {
    public void dessiner(Graphics g);
    public void setColor(Color c);
    [...]
}

class Test {
    public static void main(String args[]) {
        Graphics g = [...];
        Dessinable[] tab = new Dessinable [3];

        tab[0] = new Dessinable() { // Une belle classe anonyme pour notre
bonhomme de neige
            private Color color;
```

```
            public void setColor(Color c) {
                this.color = c;
            }
            public void dessiner(Graphics g) {
                g.drawOval(...);
                [...]
            }
        };
        tab[1] = new Rectangle(...);
        tab[2] = new PangolinDessinable(...);
        for(Dessinable d : tab) {
            d.dessiner(g);
        }
    }
}
```

Soyons clair : la syntaxe `new Dessinable()` ne signifie pas que l'on instancie directement une interface (ce qui serait contradictoire avec le fait qu'une interface ne contient que des méthodes abstraites). Il s'agit ici d'un raccourci d'écriture qui condense la définition d'une classe réalisant l'interface `Dessinable` et son instanciation en une seule ligne d'instructions. D'ailleurs, ce raccourci n'est possible que parce que l'on spécifie explicitement le code des méthodes abstraites lors de l'instanciation.

La contrepartie de ce raccourci d'écriture est que cette classe anonyme ne peut être formellement instanciée qu'une seule fois, lors de sa définition.

Un exemple d'interface

Vous découvrirez progressivement que la notion d'interface a beaucoup d'usages en POO.

L'exemple qui suit montre qu'une méthode (un algorithme) peut prendre en paramètre des références du type d'une interface. Il suffit que le corps de cette méthode n'utilise que les méthodes déclarées dans l'interface. Dès que cela est respecté, en effet, on est sur que tout se passera bien : tout objet qui sera passé en paramètre à notre méthode sera une instance d'une classe qui implémente l'interface et qui, en conséquence, possédera nécessairement toutes les méthodes utilisées. On est donc certain, dès la compilation, que ces méthodes pourront bien être exécutées.

L'important dans cet exemple n'est pas l'algorithme du tri par insertion, mais :

- la notion d'interface et son utilisation dans le traitement générique de la méthode trierParInsertion() de la classe Tri
- l'usage du polymorphisme : des objets de types dynamiques variables sont manipulés au moyen de références (type statique) de type InterfaceComparable.
- le fait que, grace au polymorphisme, l'algorithme de tri peut s'appliquer indifféremment à n'importe quel type d'objets dès lors que ces objets implémentent l'interface InterfaceComparable.

Ainsi, on écrit une fois pour toutes l'algorithme de tri... pour trier n'importe quels types d'objets "comparables" entre eux !

```
/** Toute classe qui implémente l'interface InterfaceComparable
 *  L'interface InterfaceComparable impose donc l'existence d'une
 *  une relation d'ordre sur les classes qui l'implémente.
 */
interface InterfaceComparable {
    // Signature de la relation de comparaison
    // Retourne true si "this <= other"
    boolean infEgal(Object other);
}

class Tri {
    /**
     * tri par insertion d'un tableaux d'objets implantant l'interface
InterfaceComparable.
     * Cette méthode est capable de trier n'importe quel tableau d'objets
     * dès lors que leur classe implémente l'interface InterfaceComparable
!
     */
    public static void trierParInsertion(InterfaceComparable[] t) {
        InterfaceComparable aux;
        int i;
        for (int k = 1; k < t.length; k++) {
            aux = t[k];
            i = k - 1;
            while (i >= 0 && (!t[i].infEgal(aux))) {
                t[i + 1] = t[i];
                i--;
            }
            t[i + 1] = aux;
        }
    }
}

/**
 * exemple de classe implémentant l'interface InterfaceComparable
 * La classe Entier encapsule une valeur de type int
 * et définit une relation d'ordre entre objets de type Entier
 */
class Entier implements InterfaceComparable {

    int valEntier;

    public Entier(int i) {
        valEntier = i;
    }

    @Override
    public boolean infEgal(Object x) {
        if (x.getClass() != this.getClass()) {
            return false;
        }
        // Le cast explicite est nécessaire pour le compilateur...
        // Mais, grâce au if, on est sur qu'il ne génèrera pas une
exception
        Entier e = (Entier) x;
        return this.valEntier <= e.valEntier;
    }

    public String toString() {
        return "Entier " + valEntier;
    }
}

/**
 * Autre exemple de classe implémentant l'interface InterfaceComparable
```

```java
 * Une autre Voiture est "supérieure" à la Voiture this si sa puissance est
 * supérieure à la puissance de this.
 */
class Voiture implements InterfaceComparable {

    double puissance;

    public Voiture(double puissance) {
        this.puissance = puissance;
    }

    @Override
    public boolean infEgal(Object x) {
        if (x.getClass() != this.getClass()) {
            return false;
        }
        // Le cast explicite est nécessaire pour le compilateur...
        // Mais, grâce au if, on est sur qu'il ne génèrera pas une
exception
        Voiture e = (Voiture) x;
        return this.puissance <= e.puissance;
    }

    public String toString() {
        return "Voiture " + puissance;
    }
}

/** Et pourquoi ne pas "comparer" des Figures entre elles ?
 * Considérons, par exemple qu'une Figure est "inférieure" à une autre
Figure si sa surface est
 * intérieure à la surface de cette autre Figure...
 */
abstract class Figure implements InterfaceComparable {
    [... notre classe figure ...]

    abstract public double calculerSurface();

    @Override
    public boolean infEgal(Object other) {
        if (! other instanceof Figure) {
            return false;
        }
        return this.calculerSurface() <= ((Figure)
other).calculerSurface();
    }
}

class Cercle extends Figure {
    private double rayon;
    [...]
    public double calculerSurface() {
        return Math.PI * rayon * rayon ;
    }
}


public class ExempleTri {
    public static void main(String args[]) {
        InterfaceComparable[] tab = new InterfaceComparable[5];
        tab[0] = new Entier(1);
        tab[1] = new Entier(9);
        tab[2] = new Entier(5);
        tab[3] = new Entier(10);
        tab[4] = new Entier(8);

        Tri.trierParInsertion(tab);
        for (InterfaceComparable c : tab) {
            System.out.println(c);
        }
```

```
        System.out.println("--------");

        tab = new InterfaceComparable[8];
        tab[0] = new Voiture(1.5);
        tab[1] = new Entier(9);
        tab[2] = new Entier(5);
        tab[3] = new Entier(10);
        tab[4] = new Voiture(5.1);
        tab[5] = new Voiture(8.5);
        tab[6] = new Cercle(1, 1, 10);
        tab[7] = new Cercle(2, 3, 100);

        Tri.trierParInsertion(tab);
        for (InterfaceComparable c : tab) {
            System.out.println(c);
        }
    }
}

// Ce programme affiche :
//Entier 1
//Entier 5
//Entier 8
//Entier 9
//Entier 10
//--------
//Entier 5
//Entier 9
//Entier 10
//Voiture 1.5
//Voiture 5.1
//Voiture 8.5
//Cercle 10
//Cercle 100
```

Notez que cet exemple a vocation pédagogique. Dans la vraie vie, Java dispose déjà d'une interface nommée `Comparable` et de méthodes qui implantent les algorithmes de tris usuels entre objets `Comparable`.